Aim

To implement a Shift-Reduce Parser in C for the grammar:

```
E → E + E
E → E * E
E → ( E )
E → id
```

Algorithm

1. **Start the program.**
2. **Read the input string** (e.g., `id+id$`).
3. Initialize:
   - Stack with only `$`.
   - Input pointer at 0.
4. **Repeat until end of input:**
   - Shift the next symbol from input to stack.
   - After each shift, attempt reductions based on grammar rules:
     - Replace `id` with `E`.
     - Reduce patterns like `E+E`, `E*E`, or `(E)` to `E`.
   - Print the current stack, input, and action (shift or reduce).
5. **At end of input:**
   - If stack is exactly `$E` and input is at `$`, accept.
   - Otherwise, reject.
6. **Stop the program.**

Program Code

```
#include <stdio.h>
#include <string.h>

char input[100], stack[100] = "$";
int top = 0, ip = 0;

void push(char ch) {
```

```
    stack[++top] = ch;
    stack[top + 1] = '\0';
}

void pop(int count) {
    top -= count;
    stack[top + 1] = '\0';
}

int check_reduce() {
    if (top >= 2 && stack[top - 1] == 'i' && stack[top] == 'd') {
        pop(2);
        push('E');
        printf("Reduce E → id\\n");
        return 1;
    }

    if (top >= 3 && stack[top - 2] == '(' && stack[top - 1] == 'E' && stack[top] == ')') {
        pop(3);
        push('E');
        printf("Reduce E → (E)\\n");
        return 1;
    }

    if (top >= 3 && stack[top - 2] == 'E' && stack[top - 1] == '+' && stack[top] == 'E') {
        pop(3);
        push('E');
        printf("Reduce E → E+E\\n");
        return 1;
    }

    if (top >= 3 && stack[top - 2] == 'E' && stack[top - 1] == '*' && stack[top] == 'E') {
        pop(3);
        push('E');
        printf("Reduce E → E*E\\n");
        return 1;
    }

    return 0;
}

void print_status(const char *action) {
    printf("%-15s %-15s %s\\n", stack, input + ip, action);
}
```

```
int main() {
    printf("Enter the input string (ending with $): ");
    scanf("%s", input);

    printf("\\n%-15s %-15s %s\\n", "Stack", "Input", "Action");
    printf("-----------------------------------------------\\n");

    while (input[ip] != '\0' && input[ip] != '$') {
        if (input[ip] == 'i' && input[ip + 1] == 'd') {
            push('i'); push('d');
            print_status("Shift id");
            ip += 2;
        } else {
            push(input[ip]);
            char act[20] = "Shift ";
            act[6] = input[ip];
            act[7] = '\0';
            print_status(act);
            ip++;
        }

        while (check_reduce()) {
            print_status("");
        }
    }

    while (check_reduce()) {
        print_status("");
    }

    if (strcmp(stack, "$E") == 0 && input[ip] == '$') {
        print_status("Accept");
    } else {
        print_status("Reject");
    }

    return 0;
}
```

Sample Input and Output

Input

```
id+id$
```

Output

```
Stack           Input           Action
-----------------------------------------------
$id             +id$            Shift id
$E              +id$            Reduce E → id
$E+             id$             Shift +
$E+id           $               Shift id
$E+E            $               Reduce E → id
$E              $               Reduce E → E+E
$E              $               Accept
```